Studio Nx

Backend(FastAPI/Python) 46 — withdraw()

2026-10-12

Backend(FastAPI/Python) 46 — `withdraw()`

항목	내용
파일	`studionx/backend/app/services/user_service.py`
작업	P1-09 비밀번호 변경·탈퇴: 회원 탈퇴(소프트 삭제) (PRD 01 FR-108)
작성	Claude (AI), 2026-10-07, 사용자 검토
이전 · 다음	45 `change_password()` · 47 `PUT /users/me/password`

1. 하는 일

비밀번호를 확인한 뒤 회원을 **탈퇴 상태**로 바꿉니다. 회원 행은 지우지 않으며(소프트 삭제), 프로필 사진은 지웁니다.

바뀌는 것	바뀌지 않는 것
`status` → `DELETED`	이메일과 닉네임 (3일 동안 재가입을 막는 데 씁니다)
`deleted_at` → 지금 시각 (UTC)	회원 행 자체
`profile_image_path` → `None`, 사진 파일 삭제	내가 OWNER인 프로젝트

상황	결과
비밀번호가 맞음	위 표처럼 바뀝니다
비밀번호가 틀림	400 `WRONG_PASSWORD`이고, 아무것도 바뀌지 않으며 사진도 그대로입니다
저장 실패	오류를 그대로 전달하고, 사진 파일도 남습니다

탈퇴한 뒤에 일어나는 일 (다른 함수들이 이미 처리합니다)

무엇	결과	막는 곳
로그인	401 (이유는 숨깁니다)	29 `authenticate`
남아 있는 Access Token (최대 30분)	401	06 `get_current_user`
자동 로그인 쿠키로 재발급	401	30 `get_active_user`
3일 안에 같은 이메일이나 닉네임으로 가입	409 `REJOIN_TOO_SOON`	19, 20 (P1-04)
사진 주소	404	40
3일 뒤	이메일, 닉네임과 남은 개인 정보를 비웁니다	49번 보관 기간 정리

2. 언제 쓰나

쓰는 곳	
`DELETE /users/me` (48)	본문 `{"password": "…"}`를 받아 이 함수를 부르고, 리프레시 쿠키를 지운 뒤 204로 응답합니다
화면	userInfo.html의 Delete account → FIX-01 `promptDialog` (비밀번호만 받고 체크박스는 없음, P1-10)

3. 코드

import (파일 맨 위)

```
import re
from datetime import UTC, datetime
from pathlib import Path, PurePosixPath
…
```

`from datetime import UTC, datetime`을 이번에 추가했습니다. `UserStatus`는 40번에서 이미 넣었습니다.

함수

```
def withdraw(db: Session, user: User, password: str) -> None:
    check_current_password(user, password)
    user.status = UserStatus.DELETED
    user.deleted_at = datetime.now(UTC).replace(tzinfo=None)
    delete_profile_image(db, user)
```

문법 풀이

import 문법	뜻
`from datetime import UTC, datetime`	지금 시각(UTC 기준)을 구할 때 씁니다

식	뜻
`datetime.now(UTC).replace(tzinfo=None)`	지금 UTC 시각에서 시간대 표시를 떼어 냅니다. DB의 `DATETIME` 칸은 시간대를 저장하지 않고 항상 UTC로 쓰기 때문입니다 (P1-04와 같음)
`user.status = …`, `user.deleted_at = …`	값만 바꿉니다. 아직 DB에 저장되지 않았습니다
`delete_profile_image(db, user)`	39번 함수입니다. 사진 경로를 비우고 **`commit`**하므로, 위의 두 줄도 이 `commit`으로 함께 저장됩니다. 저장한 뒤 파일을 지웁니다

4. 흐름

1. 비밀번호를 확인하고, 틀리면 400으로 응답합니다 (아무것도 바뀌지 않습니다).
2. 상태와 탈퇴 시각을 바꿉니다 (아직 저장하기 전입니다).
3. 39번을 부릅니다. 사진 경로를 비우고 한 번에 저장한 뒤, 사진 파일을 지웁니다.

5. 왜 이렇게

선택	이유
39번을 **마지막에** 불러서 저장을 한 번으로 처리	템플릿이 "commit은 마지막에 한 번"을 요구했습니다. 상태만 저장되고 사진은 남는(또는 그 반대인) 반쪽짜리 탈퇴가 생기지 않습니다. 39번을 만들 때 이렇게 쓰일 것을 미리 정해 두었습니다 (39 §5)
이메일과 닉네임을 바로 비우지 않음	2026-10-03 결정입니다. 탈퇴하고 같은 이메일로 곧바로 다시 가입해서 제재나 기록을 피하는 것을 3일 동안 막습니다
프로젝트를 남김	2026-10-03 결정입니다. 함께 작업한 사람의 곡이 사라지면 안 되기 때문입니다. 다른 사람에게는 'Deleted user'로 보입니다 (Phase 6, 추적 방지 ①)

선택	이유
리프레시 쿠키는 라우터에서 지움	쿠키는 **응답**에 붙는 것이라서, DB를 다루는 서비스와 나눴습니다. 쿠키를 지우지 못하더라도 30번이 재발급을 막습니다
사진 파일은 바로 지움	재가입을 막는 데 쓰이지 않으므로 3일을 기다릴 이유가 없습니다

6. 시험 (`tests/test_user_service.py`에 4개를 추가해서 모두 74개, 실제 MySQL과 임시 폴더로 실행)

시험	확인한 것
탈퇴	DB에 `DELETED`와 지금 시각이 저장되고, 이메일과 닉네임은 그대로이며, 사진 경로는 `None`이 되고 파일이 지워지며, OWNER 프로젝트는 그대로입니다
비밀번호가 틀림	400이고, 상태는 `ACTIVE`이며 사진 파일도 그대로입니다
탈퇴한 뒤	로그인 401, 재발급 401, 같은 이메일과 닉네임은 `REJOIN_TOO_SOON`입니다
저장 실패	오류가 그대로 전달되고, 사진 파일이 남습니다

전체 시험 332개가 통과했고, `ruff` 검사도 통과했습니다. (P1-08 사이트 시험이 남긴 빈 `storage/profiles/` 폴더 내용을 지워서, 이제 `.gitkeep`만 남아 있습니다.)